

USER_GUIDE

User Guide

Installation

go get digital.vasic.llmorchestrator

Or clone and build:

```
git clone https://github.com/vasic-digital/LLMOrchestrator.git
cd LLMOrchestrator
go build ./...
```

Configuration

1. Copy .env.example to .env
2. Set agent binary paths (ensure CLIs are installed)
3. Configure API keys for each provider
4. Adjust pool size and timeouts as needed

Usage as Library

Basic Agent Pool

```
package main

import (
    "context"
    "digital.vasic.llmorchestrator/pkg/agent"
    "digital.vasic.llmorchestrator/pkg/adapter"
)

func main() {
    pool := agent.NewPool()

    // Register agents.
```

```

    claude := adapter.NewClaudeCodeAgent("claude-1",
        adapter.AdapterConfig{
            BinaryPath: "/usr/local/bin/claude",
            Timeout:    60 * time.Second,
        })
    pool.Register(claude)

    // Acquire an agent.
    ctx := context.Background()
    a, _ := pool.Acquire(ctx,
        agent.AgentRequirements{NeedsVision: true})
    defer pool.Release(a)

    // Start and send.
    a.Start(ctx)
    resp, _ := a.Send(ctx, "Analyze this screen")
    fmt.Println(resp.Content)
    a.Stop(ctx)
}

```

File-Based Communication

```

ft, _ := protocol.NewFileTransport("/tmp/helix-session-001")
ft.WriteToInbox(protocol.FileMessage{
    ID: "task-1", Type: "instruction", Content: "Navigate to
    settings",
})
results, _ := ft.ReadFromOutbox()

```

Response Parsing

```

p := parser.NewParser()
result, _ := p.Parse(rawLLMOutput)
fmt.Println(result.Actions)
fmt.Println(result.Issues)

```

Standalone CLI

```

# Start orchestrator.
HELIX_ENV_FILE=.env go run cmd/orchestrator/main.go

# Version.
go run cmd/orchestrator/main.go version

```

Troubleshooting

- **Agent fails to start:** Verify binary path exists and is executable

- **Circuit breaker opens:** Check agent logs, increase timeout or retry count
- **Path traversal error:** File names cannot contain .. or be absolute paths